

5.7 - Canary release e feature flags: exposição progressiva e desacoplamento

Implantar o código não significa liberar a funcionalidade

Canary release, validação com tráfego real

Pequena fração de usuários testa a nova versão antes de todos

- **Nova versão recebe fração pequena do tráfego, tipicamente entre 1% e 5%**
 - Roteamento controlado por load balancer, service mesh ou gateway
- **Monitoramento intenso compara métricas da canary contra a versão estável**
 - Erros, latência, throughput, alertas, todos analisados em janela curta
- **Se métricas degradam, rollout é abortado antes de afetar a maioria**
 - Automação de promoção é possível, mas exige SLOs e baseline confiáveis
- **Se métricas se mantêm saudáveis, fração aumenta progressivamente até 100%**
 - 5%, 25%, 50%, 100% é uma curva comum, ajustável por confiança no time

Os 5% que salvaram os outros 95%

- **Novo algoritmo de roteirização promete reduzir tempo médio de entrega em 15%**
 - Funciona perfeitamente em staging com dados sintéticos de até 200 pontos

LUNALOG

O time de rotas da LunaLog desenvolveu um novo algoritmo de roteirização ao longo de quatro meses. Em staging, com dados sintéticos, reduzia o tempo médio em 15%. Em vez de promover direto para produção, Ana decidiu por canary release, 5% do tráfego real por 48 horas. Na sexta hora do canary, o monitoramento identificou degradação em cidades com mais de 500 pontos de entrega. O algoritmo entrava em loop combinatório com volumes reais, comportamento que dados sintéticos nunca tinham reproduzido. O canary foi abortado, o bug isolado, corrigido em três dias, e relançado em novo canary. O cenário sem canary teria afetado 100% das transportadoras simultaneamente em uma sexta-feira.

Feature flags, separando deploy de release

O código chega à produção sem estar ativo para os usuários

- **Funcionalidade é envolvida em condicional controlada por configuração externa**
 - Configuração lida em runtime, sem novo deploy, sem reinício de serviço
- **Permite ligar e desligar funcionalidades de forma cirúrgica**
 - Por usuário, por região, por porcentagem, por característica de tráfego
- **Deploy se torna um evento técnico, release se torna decisão de produto**
 - Time de produto controla quando os usuários veem a funcionalidade
- **Kill switch desliga funcionalidade problemática em produção sem novo deploy**
- **Ferramentas dedicadas, LaunchDarkly, Unleash, ConfigCat, ou implementação própria simples**

Deploy não é release, e isso muda tudo

Deploy (técnico)

- Código novo chega ao ambiente de produção
- Pipeline automatizado, sem cerimônia
- Acontece várias vezes ao dia
- Responsabilidade do time de engenharia
- Risco controlado por canary e flags

Release (produto)

- Funcionalidade é exposta a usuários reais
- Decisão coordenada com marketing e suporte
- Acontece quando o negócio decide
- Responsabilidade compartilhada com produto
- Reversível por flag, sem novo deploy

A armadilha das flags permanentes

- **Feature flags têm custo de manutenção, código com muitos branches vira ilegível**

Flag criada para experimento de duas semanas que continua viva dois anos depois é dívida técnica oculta. O código carrega ramos mortos, testes precisam cobrir combinações que ninguém mais usa, e ninguém sabe quem ainda depende daquela flag. Toda flag precisa de data de morte explícita no momento de criação, com responsável nomeado pela remoção.

- **Auditoria trimestral de flags, todas listadas com prazo de validade**
 - Flag sem dono em três meses vira ticket de remoção automática



Exposição progressiva com canary e desacoplamento via feature flags, o risco de cada mudança fica menor e mais controlado. Mas quando algo dá errado, e eventualmente dá, como você volta? Existem decisões de schema e de contrato que tornam o retorno tecnicamente impossível, e essas são as decisões que ninguém planeja com antecedência suficiente.

Rollback, critérios, automação e o custo de decisões irreversíveis